

HARBOR/BRIDGE/TERMINAL

A full technical dissection of the Harbor Bridge Terminal platform: the HERMIT/SPIRAL virtual terminal carried over the RAMWS hermit.vws.v2 virtual WebSocket, its bounded RAM ledger and worker isolation, the DF fabric and QVM Qnode fleet, the VB-JA21 optical-desktop client, and the Unikernel Containership UC-2.8.0 harbor yard. Every quantitative claim is anchored to a file in the supplied tree or to a run made for this note; outside literature appears only in marked side-rails.

127 / 127 recorded suite pass	120 + 6 skip re-run here, 0 fail	42.5 % W1 CPU cut (thread)	67.5 % W2 CPU cut
99.2 % W3 paste CPU cut	12 / 12 R28 results re- derived exactly	5 / 5 new property tests pass	6 findings documentation & harness defects

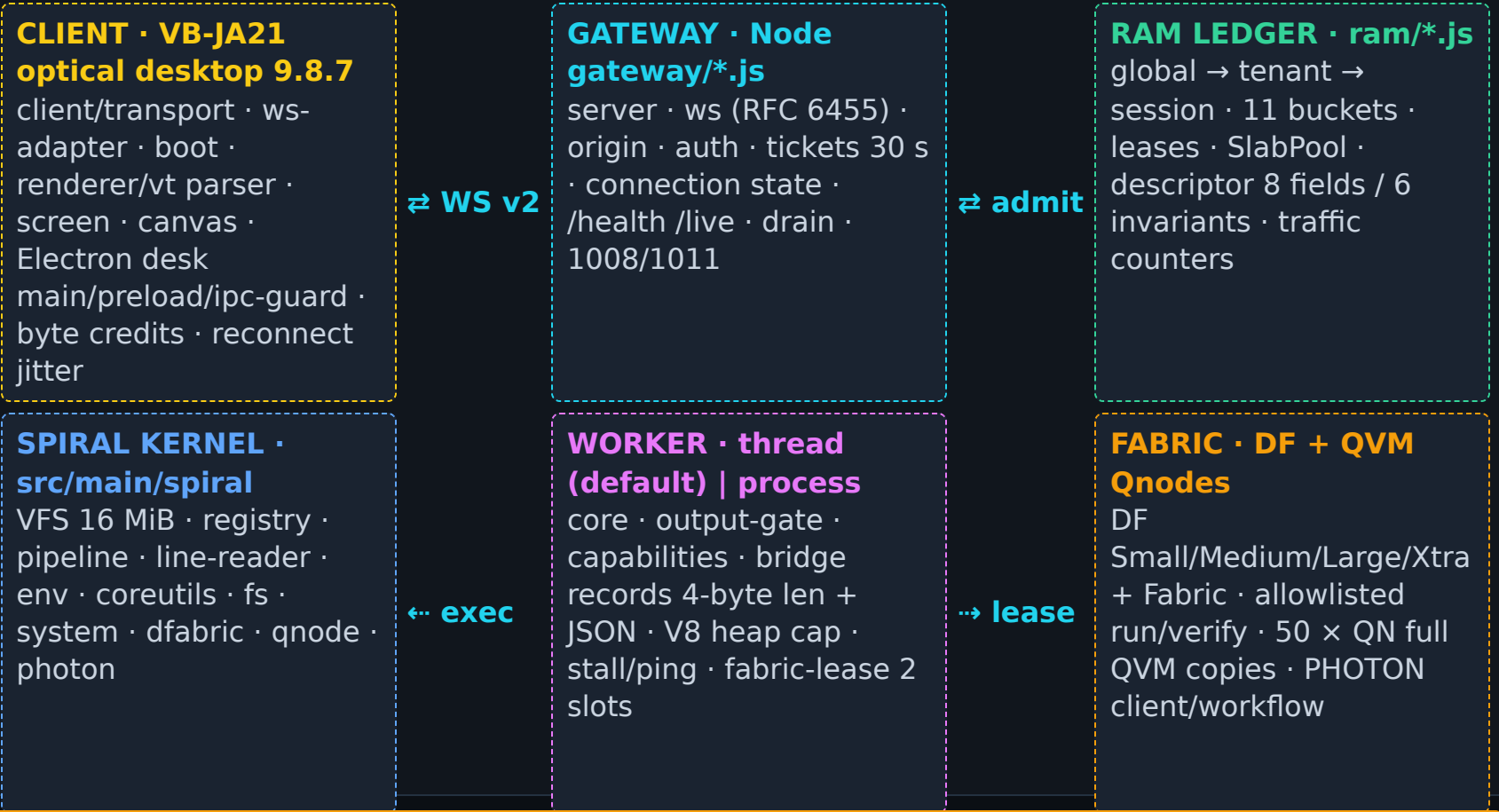
Abstract

The bridge-terminal candidate 2.0.0-ramws.1 (11 070 JS lines, 86 files) was re-tested in a clean Linux sandbox: 120 tests pass, 6 fabric tests skip because the DF containers are not in the repository, 0 fail. Five new property tests (2 000 re-chunked streams, 3 000 byte mutations, 20 000-step ledger walk) pass. All twelve preserved results of the pre-registered R28 CPU experiment were re-derived from the 240 raw run records and match the recorded points and 95 % cluster-bootstrap intervals to three decimals. A live replay of the shipped load bench gives keystroke echo p50 0.77–0.99 ms. Six defects were found — all in documentation, evidence bookkeeping or the bench harness, none in the transport path.

Reading guide

Pages	Content
01-03	scope, evidence classes, architecture and byte path
04-09	components A-G, each with discussion · explanation · applications · implications · purpose · intramodel relationships
10	test results (shipped + new property tests)
11-12	backtesting: independent re-derivation of the R28 experiment
13	live replay of the load bench
14-16	governance ledgers, findings, risk register, appendix

Figure 0.1 — layered architecture, as implemented



```
HERMIT / LK/Vexa 02 session + sign out
• SPIRAL // SESSION
}
},
"provenance": {
  "backend": "none",
  "distributed_state": "LOGICAL_DISTRIBUTED",
  "distributed_state_ceiling": "DISTRIBUTED_CLASSICAL_EMULATION",
  "execution_class": "CLASSICAL_FABRIC_OF_LOCAL_VMS",
  "host": {
    "cpu_count": 2,
    "platform": "Linux-6.18.44-fc-v37-x86_64-with-glibc2.39",
    "python": "3.11.15"
  },
  "language": "PA-LCTL",
  "network": "deny",
  "parallel_state": "SERIAL",
  "physical_distributed": false,
  "physical_parallel": false,
  "physical_gpu": false,
  "physical_release_outputs": {
    "PHYSICAL_DISTRIBUTED_QPU_EXECUTION": "BLOCKED_EXTERNAL_AUTHORITY",
    "PHYSICAL_PARALLEL_QPU_EXECUTION": "BLOCKED_EXTERNAL_AUTHORITY"
  },
  "profile": "single_process_deterministic",
  "quantum_boundary": "QUANTUM_BOUNDARY_NOT_CROSSED",
  "target_verified": false
},
"rows": 12,
"schema": "DF/FABRIC_RUN/1",
"source": "$DF_ROOT/DF_Fabric/examples/01_bell_pair.pal",
"verdict": "CROSS_NODE_DIFFERENTIAL_AGREEMENT",
"wall_s": 0.7861
}
operator@hermit ~ > |
```

Figure 0.2 — recorded evidence e/I055/02-fabric-run.png: a DF_Fabric bell-pair run reaching CROSS_NODE_DIFFERENTIAL_AGREEMENT over the socket

1.1 What was read, what was run

Item	Status
Target zip Harbor-Bridge-Terminal-main (2).zip (222.4 MB)	upload refused twice by the device bridge (size); not opened
Extracted folder Downloads\Harbor-Bridge-Terminal-main	1 138 files, 30.9 MB, mtime 2026-09-24 16:0x — the (1)/plain zip, 366 KB smaller than (2)
bridge-terminal/ (all source, tests, docs, e/, ledger outputs)	staged in full and executed
containership/ docs, conformance, edge manifest, authority, corpus	staged and parsed
containership berths (5 of 6), PHOTON control_plane, shell	listed only (sizes/counts)
ship/, hull/, hold/, pk/, edge/source	absent from the tree; uc.py cannot import unikernel.cli
QVM Qnodes (50 × ~17 MB), JA21 portable, DF containers	gitignored junctions, not present

Consequence: this note documents the (1) snapshot. The 366 KB difference to (2) is unknown; re-running this note against (2) needs the zip extracted on disk or a smaller archive.

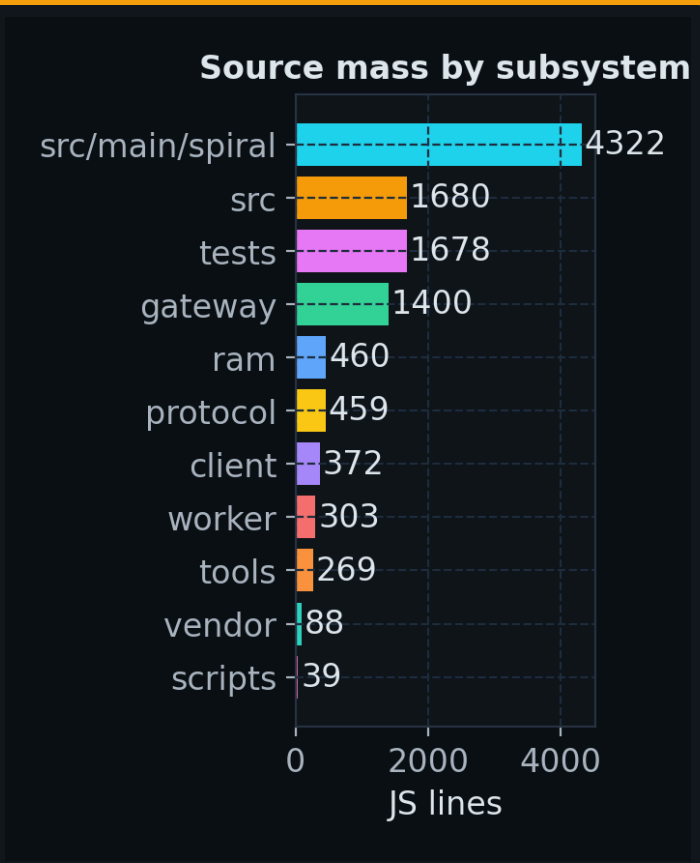


Figure 1.1 — JavaScript line mass per subsystem. SPIRAL (the kernel and its 9 command modules) is 39 % of the code; gateway + protocol + ram — the transport — is 21 %.

1.2 Evidence classes used

Class	Meaning in this note
OBSERVED-HERE	run in this sandbox on 2026-09-25: Linux 6.18.44, 2 vCPU Xeon 2.8 GHz, Node v22.22.2
RE-DERIVED	recomputed by this note from preserved raw files
RECORDED	read from a file in the tree; not re-executed
DECLARED	a design statement in docs; no test found
ABSENT	referenced but not present in the tree

1.5 Limits of this note

Single sandbox host; no Windows run of the .cmd launchers; no JA21 or Electron window launched; the containership could not be executed (ship layer absent).

1.4 Method

Files were staged from the linked computer in 16 batches, copied to a scratch tree, and executed unchanged. Nothing in the product was edited; new tests live in a separate folder. The R28 re-derivation reads only raw*.jsonl and compares against result*.json, so it tests the project's analysis code independently of its own numbers. Statistics use a fixed seed (28) so the note is reproducible.

1.3 Two products in one repository

Product	Version	Language	Role
HERMIT bridge-terminal	2.0.0-ramws.1	Node ≥ 20 / Electron 31	the "bridge": live console + virtual WebSocket
Unikernel Containership	UC-2.8.0	Python 3 (+ C/Java engines)	the "harbor yard": berths, gates, atoms
VB-JA21 Optical Desktop	9.8.7 (junction)	—	the browser the gateway opens in
QVM Qnode fleet	8.1.0-alpha seed	Python	50 independent full copies

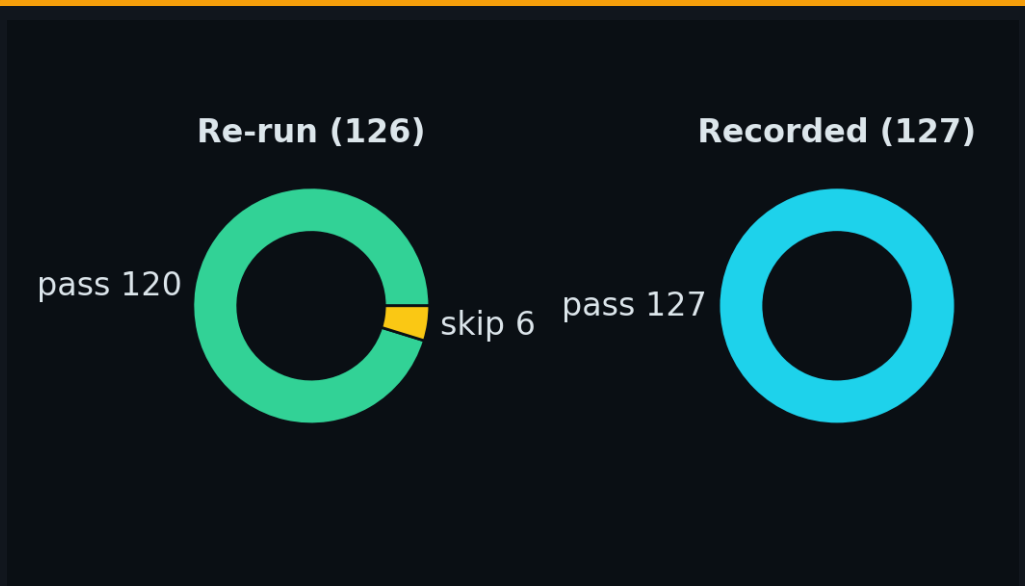


Figure 1.2 — Suite outcome: re-run here vs the recorded e/full-suite.tap (127 tests, 94.3 s). The recorded run had DF containers attached; the skip set is exactly the 6 fabric-over-WebSocket tests.

Side-rail · outside literature (corroborating only)

RFC 6455 (IETF, 2011) defines WebSocket framing, the 101 upgrade and close codes 1008 (policy violation) and 1011 (internal error) that the gateway emits. Efron & Tibshirani, An Introduction to the Bootstrap (1993) is the standard reference for the percentile interval used in R28; with 3 clusters per arm the resampling space is only 3³ × 3³ = 729 distinct draws, which is why the project itself labels its intervals "narrow only because run-to-run noise is small". Base64 (RFC 4648) expands 3 bytes to 4, the 1.333× factor the v1 protocol paid on every data frame.

2.1 The byte path, end to end (thread backend)

- Key press** in the JA21 omni-bin → `client/ws-adapter.js` encodes a **binary** WS message: byte 0 = kind 1 (input), bytes 1-6 = uint48 big-endian seq, then raw bytes ($\leq 8\,192$).
- Gateway** `gateway/ws.js` parses the RFC 6455 frame; `connection.js` checks seq = last + 1, the per-connection message rate (300/s) and the session's ledger account.
- Ledger** charges the receive buffer's capacity to *session* → *tenant* → *global* before allocation; a refusal names the scope (**EBUDGET**).
- Worker link** transfers the `ArrayBuffer` to the session's worker thread (no copy, counted as `bridge_out_transfer`); the process backend instead writes a 4-byte-length JSON record to stdin.
- SPIRAL** `LineReader` edits the line, `pipeline.js` tokenises, `registry.js` dispatches a command against the in-memory VFS.
- OutputGate** chunks output ($\leq 16\,384$ B), counts in-flight bytes and pauses the producer at $\frac{1}{2}$ of 4 MiB pending.
- Gateway** writes header + payload corked, spending client **byte credits**; at zero credit the session pauses, never drops.
- Client** VT parser → Screen → canvas renderer; renews credit per $\frac{1}{4}$ window or 128 messages.

2.2 Recorded owner decisions that shape the design

ID	Decision (docs/RAMWS.md, vws200/ARCHITECTURE.md)
D-001 / RD-03	Gateway host is Node, not PowerShell (no pwsh/.NET in build env)
D-002	Remote sessions may drive DF nodes only through an allowlist
RD-01	Profile LOCAL_VOLATILE: process loss = session loss, disclosed by <code>session.reset</code>
RD-02	Protocol v2 only; a v1 client gets HTTP 400 at handshake
RD-04	One worker thread per session with enforced V8 heap cap
RD-05	Client byte credits replace v1 output acks

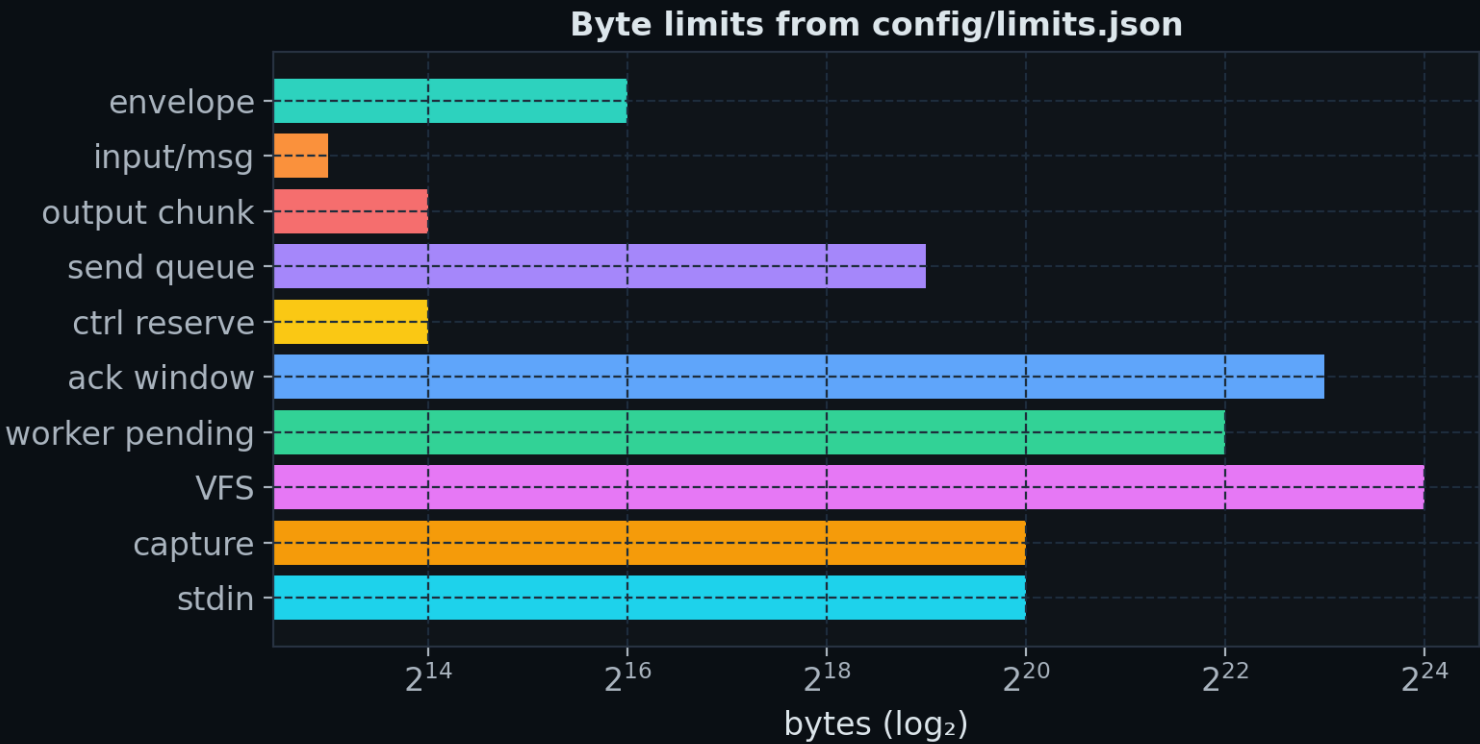


Figure 2.1 — Every byte limit in config/limits.json, log₂ scale. Note `_status`: "PROPOSED starting limits ... owner approval and exact-host measurement still required".

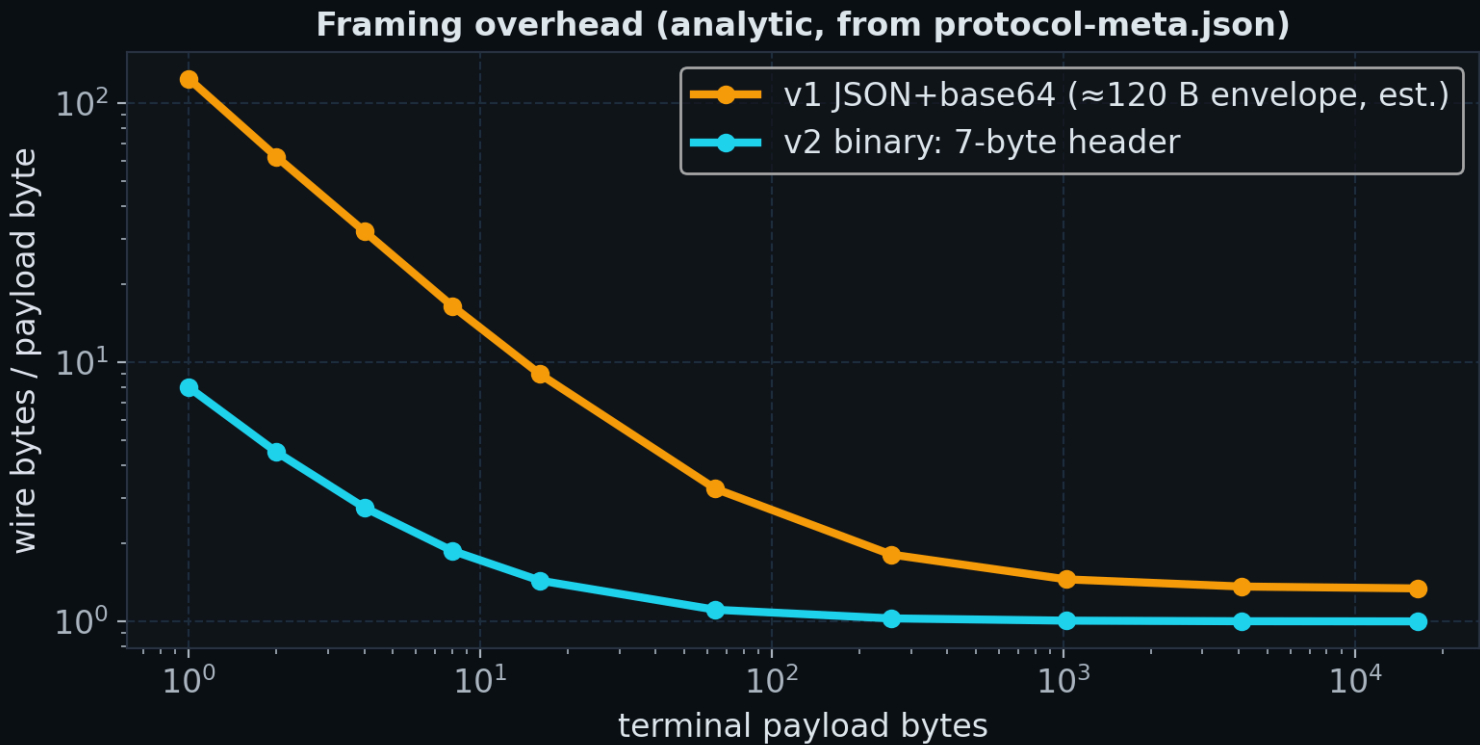


Figure 2.2 — Framing overhead per payload byte. v2 carries a fixed 7-byte header; v1 paid base64 (×1.333) plus a JSON envelope. At a 1-byte keystroke v2 sends 8 bytes of body.

A. gateway/ (9 modules, 1 400 lines)

Module	Lines	Responsibility
auth.js	81	token → principal (SHA-256 at rest)
config.js	121	env parsing, exit 78 on bad config
connection.js	355	per-socket state machine: hello → open → data → close
log.js	17	structured log
origin.js	40	Origin allowlist check
server.js	331	HTTP: /, /health, /live, drain on SIGTERM
tickets.js	50	single-use tickets, 30 s TTL, rate-limited
worker-link.js	73	thread/process worker supervisor
ws.js	332	RFC 6455 framing, masking, fragmentation, close

Discussion

The gateway is where every trust and resource decision happens: it is the only component that sees both the network and the ledger. Its design is a strict admission pipeline — Origin, ticket, principal, capacity, then protocol — so nothing expensive is done for an unauthenticated peer.

Explanation

A browser first POSTs its access token for a single-use **ticket** (≤ 60 per principal per minute, ≤ 60 failures per address per minute). The WebSocket upgrade must present that ticket within 30 s. After 101, the server sends **hello** with the epoch (random per launch), limits and capabilities; control messages are strict JSON text, data are binary.

Applications

Local operator console on 127.0.0.1:10000 (START_HARBOR.cmd); a Render.com service (deploy/render.yaml, Dockerfile) — the latter is task R26, **BLOCKED** (no deploy target from the build environment).

Implications

Because data frames carry no session id, a client can only address its own session: cross-connection injection is impossible by construction (tests/security/authz.test.js, 9 tests). The unauthenticated **/live** endpoint exposes counters but no payloads — a residual risk the project itself lists.

Purpose

Turn an in-process terminal (HERMIT desk) into a bounded, multi-session network service without adding an OS shell: the remote surface is the SPIRAL virtual command set plus an allowlisted fabric.

Intramodel relationships

Consumes **limits.json** and the **ledger** (admission), spawns **workers**, speaks **protocol v2** to the **client**, exposes fabric capability per principal. Heartbeat (20 s interval, 10 s window) and worker ping/stall (5 s / 15 s) close the loop.

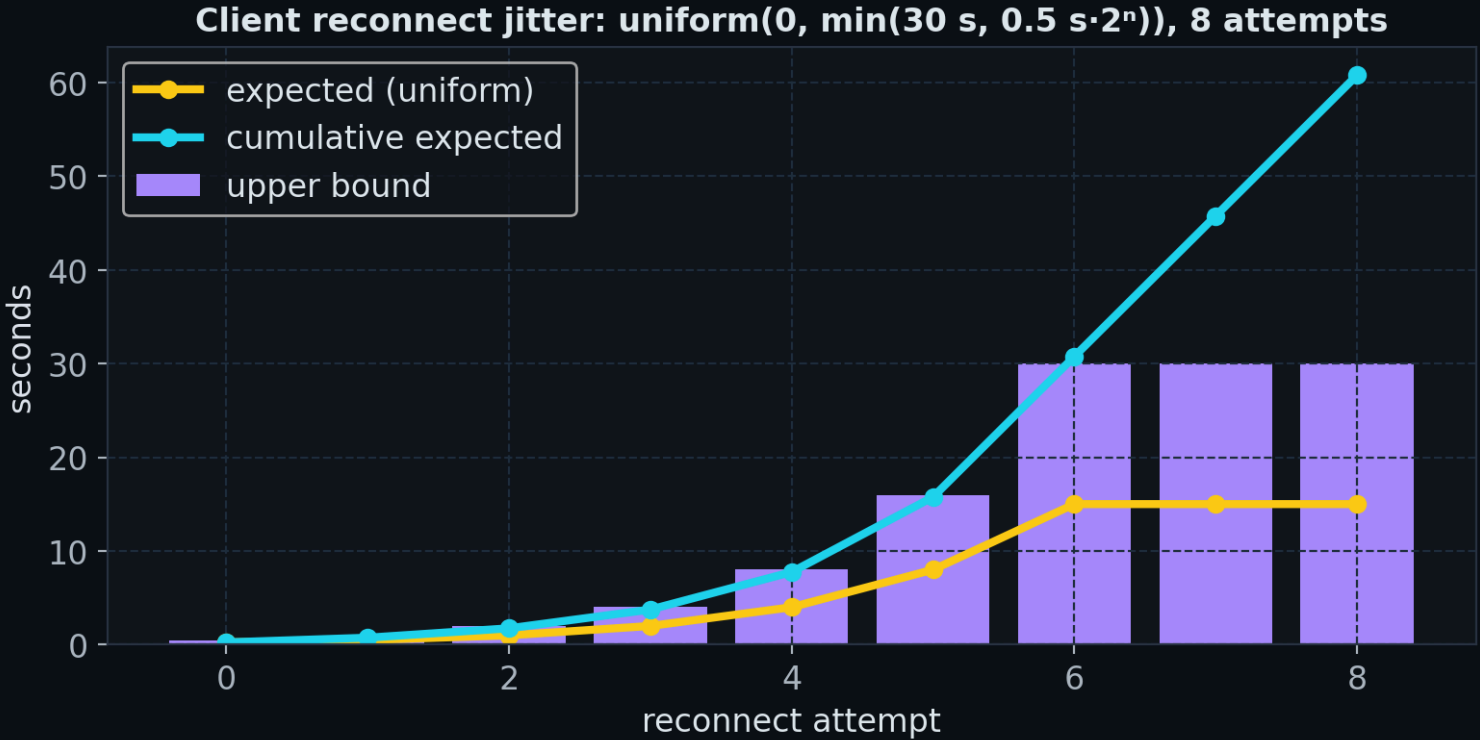


Figure 3.1 — Client reconnect policy (limits.json "reconnect"): full jitter, 8 attempts, fresh ticket each time, no input replay. Expected total wait before giving up ≈ 51 s.

A.2 Close and refusal semantics (tests/gateway/ws-conformance, 15 raw-socket tests)

Condition	Outcome
v1 subprotocol / missing ticket	HTTP 400 / 401 before upgrade
Bad Origin	403
seq gap, credit overflow, unknown type	close 1008
worker crash / kill -9	exactly one <code>session.exit{worker_failure}</code> , close 1011, capacity released
peer never answers heartbeat	close 1008 within interval + window
SIGTERM	drain within 20 s budget, <code>service.draining</code> , exit 0
malformed PORT env	refuse to start, exit 78

All seven rows correspond to named tests that passed in the re-run (chaos/lifecycle 9, chaos/review-fixes 11).

B.1 Binary data frame (protocol/protocol-meta.json)

byte 0 kind 1 in · 2 out	bytes 1-6 seq uint48 big-endian	bytes 7... raw terminal bytes 1...8 192 in · 1...16 384 out
--------------------------------	------------------------------------	--

Zero-length payload rejected (BAD_MESSAGE); seq starts at 1, +1 per direction; the connection ends before $2^{48}-1$ ($2^{48} < 2^{53}$, exact in a JS Number).

Direction	Control messages (strict JSON text, ≤ 65 536 B envelope)
client → server	session.open · terminal.resize · terminal.signal · session.close · flow.credit · heartbeat.ping/pong
server → client	hello · session.reset · session.opened · request.ack · input.ack · session.exit · heartbeat.ping/pong · service.draining · error

B.2 Gateway ↔ worker bridge records (protocol/bridge.js)

Direction	Record types
gateway → worker	open · input · resize · signal · close · pause · resume · ping · consumed
worker → gateway	ready · opened · output · exit · pong · diag · executed

Record = 4-byte big-endian length + UTF-8 JSON, body 1...65 536 B, validated against bridge.schema.json on both encode and decode; a partial record older than 5 s fails with BRIDGE_ASSEMBLY_TIMEOUT unless the reader paused deliberately.

Discussion

v2 separates what must be parsed (control) from what must only be moved (terminal bytes). This single decision removes base64, JSON parse and a session id from every keystroke and every output chunk.

Explanation

Every data frame is bound to the authenticated connection; ordering is proved by a contiguous seq. input.ack distinguishes ACK_ACCEPTED (queued) from ACK_EXECUTED (the line ran), so after a crash the client learns exactly how many inputs have an unknown outcome.

Applications

Interactive shells, bulk log streaming (W2: 6.2 MB in 0.53 s here), paste of long lines (W3), and fabric run output that is large JSON.

Implications

Schema strictness means any extension is a protocol bump. Test vectors: 12 invalid v2 messages, raw invalid frames, valid set, and RAMWS cases.json — all accepted/rejected as declared (tests/protocol, 12 tests).

Purpose

Make the transport cost proportional to the bytes actually moved, and make the wire itself un-forgable across sessions.

Intramodel relationships

Codec feeds the gateway; seq and credits drive the ledger (credits bound queued_out); the bridge record format is shared by both worker backends so the process ablation in R28 isolates the backend from the protocol.

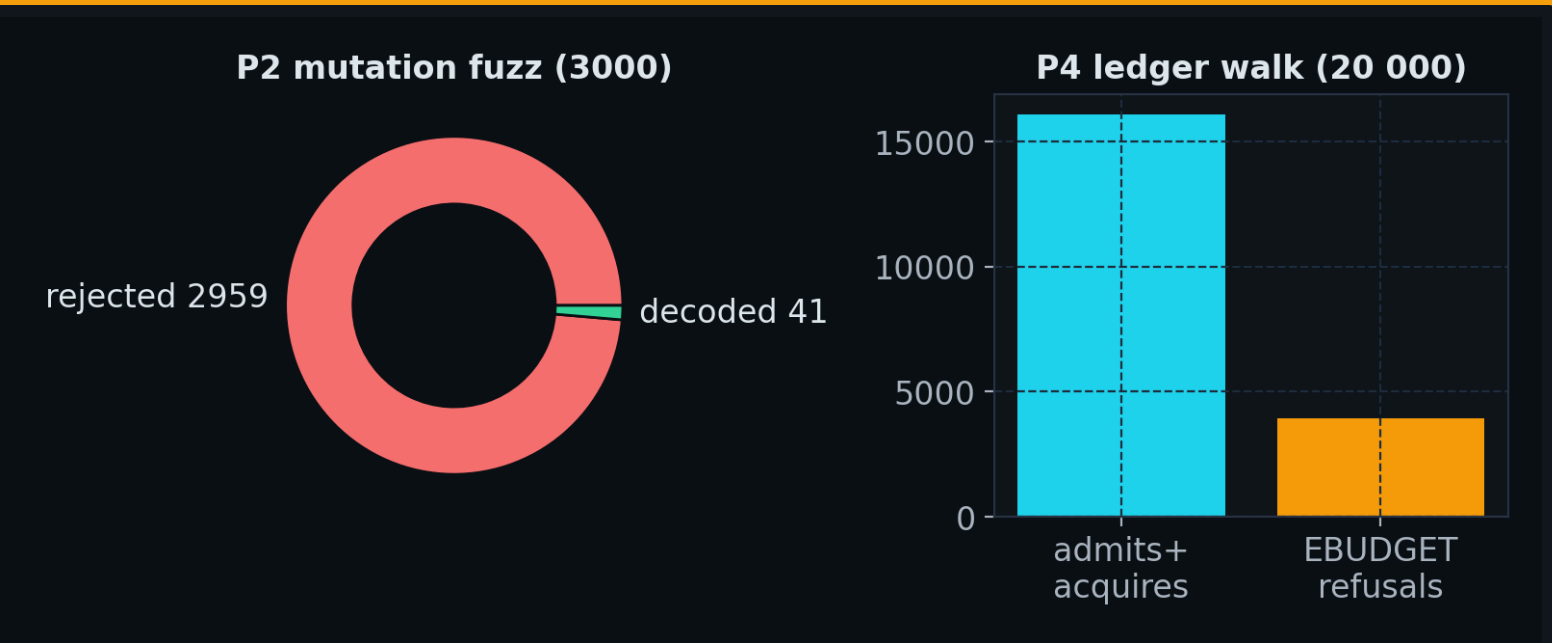


Figure 4.1 — New property tests on the bridge decoder and ledger (tests/supplementary, OBSERVED-HERE). 2 959 of 3 000 random 1-4-byte mutations are rejected; the 41 accepted mutate only string contents inside a still-valid record. No mutation threw or allocated beyond 65 536 B.

C.1 SPIRAL kernel (4 322 lines)

File	Lines	File	Lines
kernel.js	665	dfabric/locator.js	174
cmd/qnode.js	413	pipeline.js	172
cmd/dfabric.js	366	registry.js	136
vfs.js	290	photon/client.js	125
cmd/coreutils.js	269	cmd/photon.js	114
cmd/fs.js	255	dfabric/runner.js	105
cmd/system.js	254	qnodes/runner.js	101
qnodes/locator.js	251	photon/workflow.js	94
line-reader.js	247	dfabric/policy.js	78
		cmd/browser.js	76
		cmd/util.js	59
		env.js	51
		cmd/index.js	27

Discussion

The kernel is a virtual POSIX-like shell written in JavaScript: no OS process is created for ordinary commands, so a remote user cannot reach the host shell. Only the DF fabric verbs spawn a process, through a fixed interpreter and rebuilt argv.

Explanation

Per session a worker holds its own VFS (≤ 16 MiB, 1 000 nodes), environment, history (1 000 entries) and line reader (4 096 chars). **seq** streams in cooperative batches instead of building one string; regex input is capped (256 chars pattern / 16 384 line) because a catastrophic RegExp cannot be cancelled in-process — the supervisor kills only that worker.

Applications

Operator console verbs: coreutils, fs, system, dfabric (status/topology/nodes/bundles/run/verify), qnode (focus codes qn01...qn50), photon, browser (desk only).

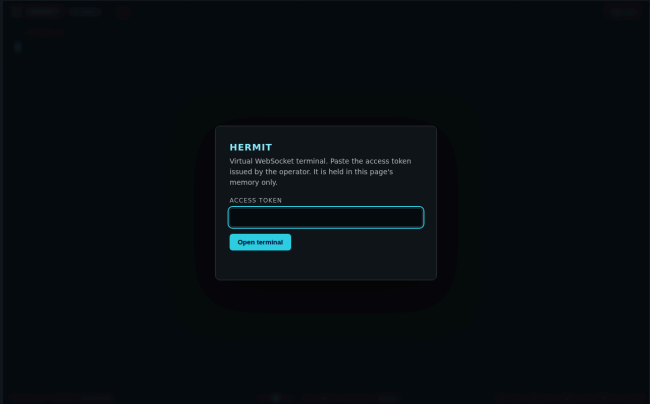


Figure 5.1 — recorded e/I055/01-signin.png — sign-in in the LK/Vexa graphite/cyan house design

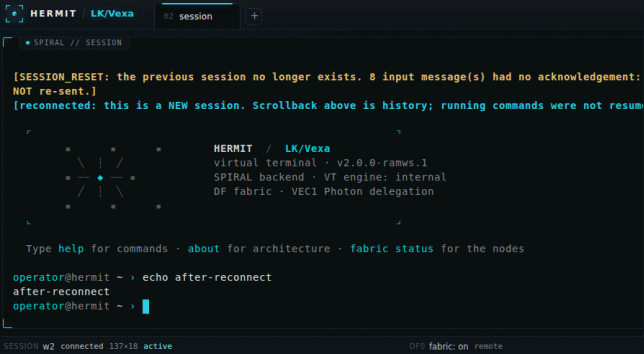


Figure 5.2 — recorded e/I055/03-reconnected.png — a new session after reconnect (no input replay by design)

Implications

Thread backend shares the gateway address space: isolation is V8-isolate level, not OS level. The docs recommend the process backend plus one container per gateway for hostile multi-tenancy.

Purpose

Keep the preserved HERMIT/SPIRAL behaviour byte-identical in a headless worker so desktop and web sessions execute the same kernel.

Intramodel relationships

Receives bridge records; OutputGate reports **consumed** per chunk (coalesced per event-loop turn — the change that moved W1 from 33 % to 42 %); fabric-lease counts concurrent DF jobs (2 slots) across sessions via pid directories in os.tmpdir()).

D.1 ram/ (5 modules, 460 lines + 2 schemas)

Construct	Behaviour (ram/ledger.js, descriptor.js)
Ledger({{globalBytes, tenantBytes}})	<code>admit()</code> charges a session reservation to global and tenant atomically; refusal changes nothing
SessionAccount	<code>acquire(bucket, capacity)</code> charges capacity (not used length) to the session and bucket cap
11 buckets	<code>receive</code> · <code>reassembly</code> · <code>queued_out</code> · <code>inflight_out</code> · <code>bridge</code> · <code>vfs</code> · <code>history</code> · <code>workspace</code> · <code>metadata</code> · <code>codec</code> · <code>worker_runtime</code>
Lease	retain/release reference counting; capacity returns on the last release; a duplicate release is counted, never refunded; generation-tagged handles
SlabPool	size classes 4 / 16 / 64 KiB, ≤ 1 MiB idle, charged once globally
<code>reconcile()</code>	proves Σ leases = reserved at every scope
Descriptor	8 kit fields (operator, buffers, formats, budget, ownership, completion, quality, backend); server-built ones skip the structural walk (trusted-shape marker)

D.2 Ledger random walk (new test P4, OBSERVED-HERE)

20 000 random admit / acquire (1 B...200 KB, 4 buckets) / release operations over ≤ 6 sessions and 2 tenants with 8 MiB global and 6 MiB tenant budgets. Every 500 steps: global ≤ budget, each session ≤ its limit, `reconcile().ok`. At the end, after releasing survivors, every session reserved = 0 and physicalBytes = 0. **3 919 refusals**, all **EBUDGET**, no other exception. The shipped suite's own walk is 4 000 steps.

Discussion

"RAM-resident" is defined narrowly and honestly in docs/RAMWS.md: session, payload and transport state live only in bounded, ledgered process memory; nothing moves computation into DRAM; the CPU still does all the work.

Explanation

Admission before allocation: the ledger answers "may I?" synchronously, before a Buffer exists. That is how a slow or hostile client is contained — its session hits its own limit and pauses, while other tenants are unaffected.

Applications

Multi-tenant hosting on small machines, capacity planning (the /live endpoint shows plan, ledger, pool and traffic), and proving the absence of unbounded queues.

Implications

Review R2-1 found a real hole before release — output held while paused sat outside the ledger (473 MB held). The fix moved the queue into the worker's OutputGate. H1 (ledger vs RSS within 10 %) was **not run**: reservation is not RSS.

Purpose

Make memory a first-class, auditable resource with a single source of truth rather than per-module ad-hoc caps.

Intramodel relationships

Charged by the gateway (`receive`, `queued_out`), the bridge (`inflight_out`), the worker (`vfs`, `history`); read by /live; its caps come from limits.json; its invariants are the kit's six (budget, lifetime, aliasing, validity, quality, transparency).

D.3 Kit tasks R01-R32 bearing on the ledger

Task	Status
R06 effective memory allowance	PASS
R07 observe residency	IN_PROGRESS
R09 bounded pools + reservation ledger	PASS
R14 reconcile full application budget	IN_PROGRESS
R15 bound VFS/history/caches	PASS
R18 copy & memory-traffic ledger	PASS
R20 end-to-end bottleneck model	IN_PROGRESS

E. Client and desktop (client/ 372 lines · src/renderer · src/main)

Discussion Two front ends share one VT engine: the Electron desk (HERMIT window with a BrowserView pane) and the web client served by the gateway, which START_HARBOR opens inside VB-JA21 rather than the system browser.	Explanation ws-adapter encodes binary frames, manages credits and reconnect jitter; transport.js abstracts WS vs IPC; renderer/vt: parser (CSI params ≤ 64 chars, values ≤ 65 535), screen (scrollback 5 000 rows / 2 M cells), canvas renderer. ipc-guard + nav-policy lock the Electron renderer to contextBridge channels and allowlisted navigation.	Applications Operator desk on Windows (NSIS + portable exe via electron-builder), browser access on loopback, accessibility live region (client/accessibility.js).
--	--	--

Implications The desk keeps a local in-process kernel; the web path goes through the gateway — the same kernel code, two trust models. Browser pane and Photon are never installed remotely.	Purpose Give the operator one bridge from which the ship, the fabric and the fleet are all visible.	Intramodel relationships Consumes protocol v2; the VT tests (9) and client-units (6) pin parser/screen behaviour; desktop-ipc/guard (4) pins the IPC allowlist.
--	---	---

F. DF fabric · QVM Qnodes · PHOTON

Discussion The fabric is where the terminal touches real execution: five DF containers (ns nm nl nx nf) and fifty full, independent QVM copies (qn01...qn50) addressable by focus code from the landing banner.	Explanation dfabric/policy.js: fixed interpreter and adapter/dfabric/cli.py, operator-bound DF_ROOT, argv rebuilt from validated tokens, bundles only from discovery, no host-path options, allowlisted env, deadlines (run 180 s, verify/build 900 s), 2 concurrency slots, host paths redacted, per-principal fabric capability; build is operator-switchable.	Applications Differential runs across nodes (Figure 0.2: 01_bell_pair.pal, verdict CROSS_NODE_DIFFERENTIAL_AGREEMENT, 12 rows, 0.786 s); Qnode load audits (fleet/QNODE_LOAD_AUDIT.md).
---	--	---

Implications Provenance in every run states physical_distributed/parallel/qpu = false and quantum_boundary NOT_CROSSED: the fabric is a classical emulation of local VMs, and says so. ~840 MB of Qnode copies are materialised on first run.	Purpose Let an authenticated operator drive heavyweight engines without giving them a host shell.	Intramodel relationships Only path from worker to OS processes; uses the fabric lease; its 6 WebSocket tests were SKIPPED here (no DF_ROOT), but PASS in the recorded tap; lease.test (3) passed.
---	---	---

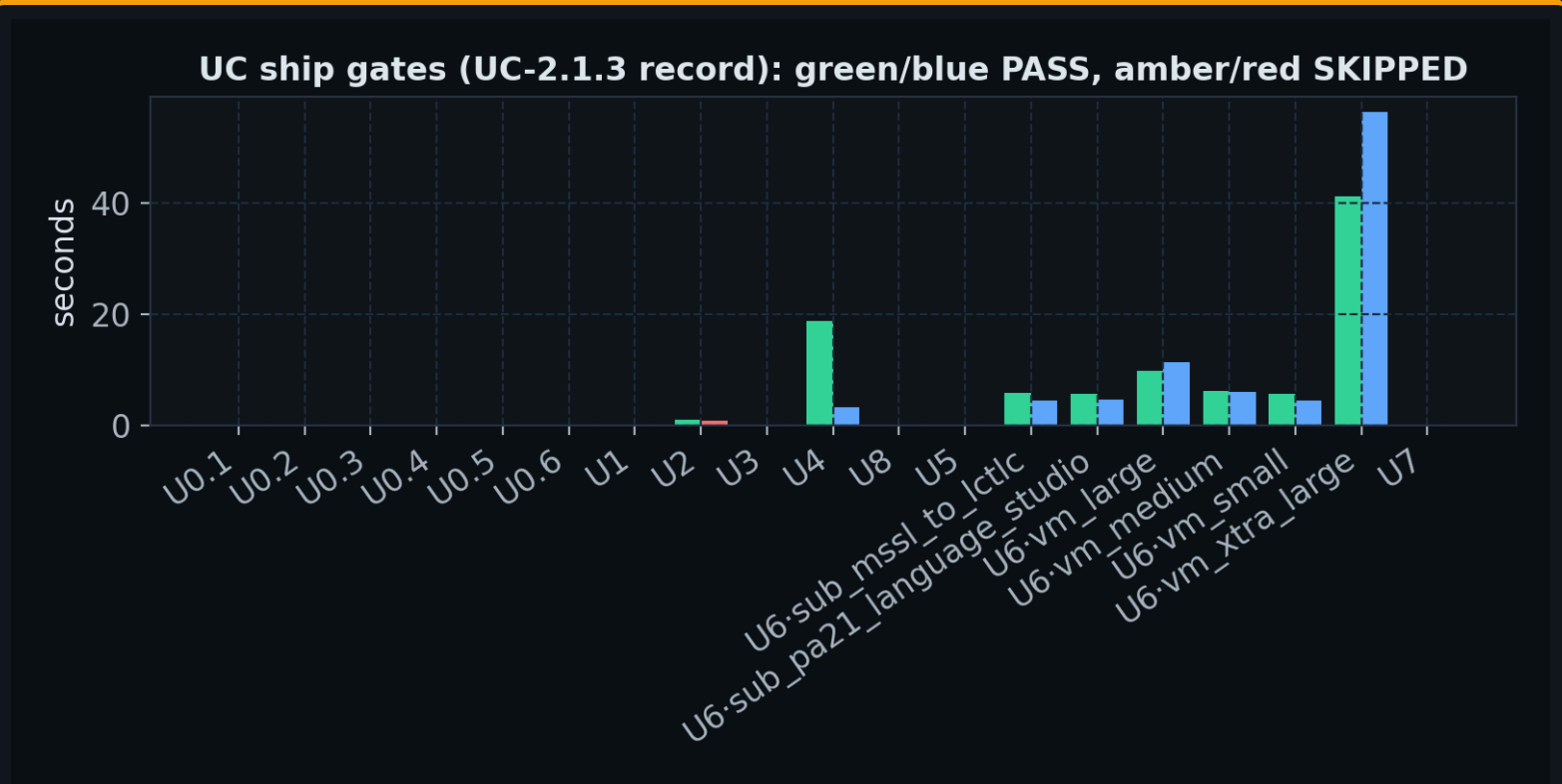


Figure 8.1 — Ship-gate durations, full vs limited host (conformance/UC_GATE_RESULTS*.json). Both verdicts PASS (17 + 2 skipped; 14 + 5 skipped). U6-vm_xtra_large dominates at 41 s. Note the record is for UC-2.1.3, dated 2026-08-17.

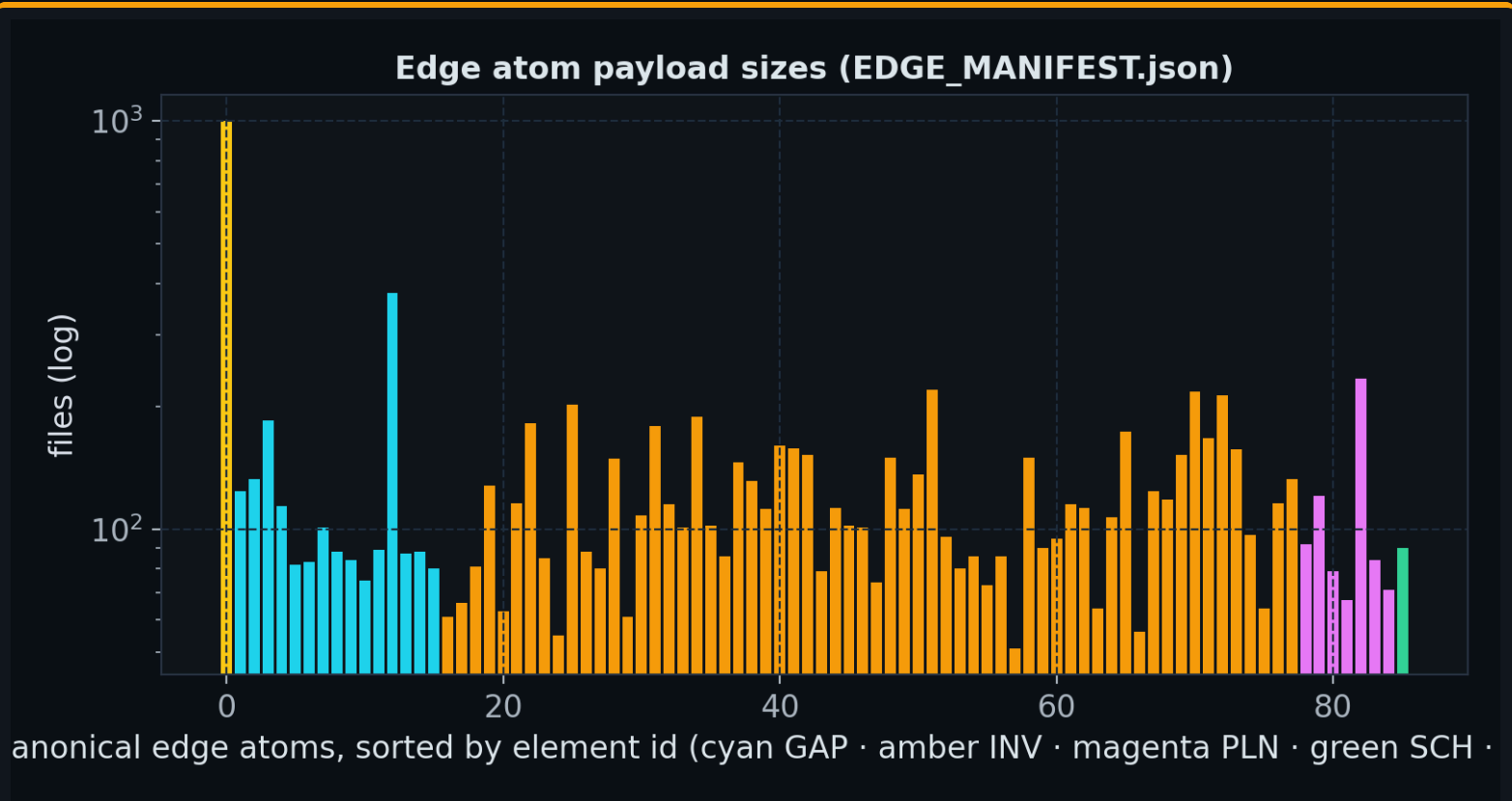


Figure 8.2 — 86 canonical edge atoms (edge/EDGE_MANIFEST.json): 10 866 files. 99 archives requested, 91 unique, 8 byte-identical duplicates, 4 variants.

Discussion

The containership is a Python-managed "ship" that sorts script cargo into six berths, each carrying DF Small/Medium/Large/Xtra/Fabric containers, verifies them with 19 ship gates and 12 berth gates, and now carries 86 edge atoms. Its own audit calls it a "locally validated, ship-layer hardened development skeleton — not a production-isolated or native-bootable unikernel platform".

Explanation

Gates U0.1-U0.6 check seals and pinned digests; U1 the sort policy (rules S0-S8, 13 examples); U2 four engines build and bind; U3/U4 the hull studio and its 135-check verifier; U5 bill of lading; U6 per-berth batteries; U7 studio tests; U8 reversible TIFF fabric codec. Limited-host mode skips what a toolchain-less Windows workstation cannot run, and says why.

Applications

Harbor yard for the bridge: EDGE.cmd status / check --deep, uc.py verify --quick, berth inspection from the terminal.

Implications

CHANGELOG records 9 833 atom tests pass / 41 fail / 46 skip across 86 suites (59 green, 27 red), all classified, no promotion to PASS, not Production GO. That evidence file (edge/evidence) is **not** in this tree, and ship/ is absent, so none of it could be re-executed here.

Purpose

Provide a reproducible, sealed, digest-pinned place to hold and exercise heterogeneous cargo with explicit, never-silent skips.

Intramodel relationships

Supplies DF containers to the fabric (hold/), PHOTON control plane to the terminal's photon verbs, and the berths the Qnodes and DF nodes are sorted into.

G.2 Gate catalogue (full host)		
Gate	Status	s
U0.1	SKIPPED	0.0
U0.2	SKIPPED	0.0
U0.3	PASS	0.001
U0.4	PASS	0.028
U0.5	PASS	0.002
U0.6	PASS	0.0
U1	PASS	0.008
U2	PASS	0.951
U3	PASS	0.12
U4	PASS	18.64

9.1 Shipped suite, re-run here

Metric	Recorded tap	Re-run
tests	127	126
pass	127	120
skip	0	6 (DF not found)
fail	0	0
duration	94.3 s	83.8 s
syntax-check	—	86 / 86 files ok

Command: `node --test tests/{protocol,worker,vt,desktop-ipc,gateway,security,fabric,chaos,ram}/*.test.js tests/ui/client-units.test.js`. The one-test difference is in the DF-dependent fabric group. `tests/ui/browser.test.js` (Playwright) is outside `npm test` and was not run.

9.2 Supplementary property tests (new)

ID	Property	Result
P1	2 000 random record streams, random 1-97 B chunking → identical records, clean end	PASS
P2	3 000 mutated frames never throw, never over-allocate	PASS (2 959 rejected)
P3	length 0, 65 537, 2 ³² −1 refused before allocation	PASS
P4	20 000-step ledger walk reconciles, returns to 0	PASS (3 919 refusals)
P5	limits.json internal consistency (7 relations)	PASS

Runtime 0.46 s. File: `tests/supplementary/hbt-exp.test.js` (delivered alongside this note).



Figure 9.1 — Test mass per file, coloured by area.

9.3 What the suite exercises, by claim

Claim	Named evidence
cross-session isolation	security/authz (9)
bounded memory under slow consumer	chaos/lifecycle #7, #8
worker failure contained	chaos/lifecycle #1-#4
exactly-once shutdown	chaos/lifecycle #11
credits lossless catch-up	ram/v2-e2e (150 000 / 200 000 lines)
RFC 6455 conformance	gateway/ws-conformance (15 raw-socket)
descriptor kit fixtures	ram/descriptor (13/13 verdicts)
VT parser bounds	vt/vt (9)
IPC allowlist	desktop-ipc/guard (4)

Not covered by any test: Render deployment (R26 BLOCKED), H1 capacity vs RSS, H4 hardware traffic counters, human security review.

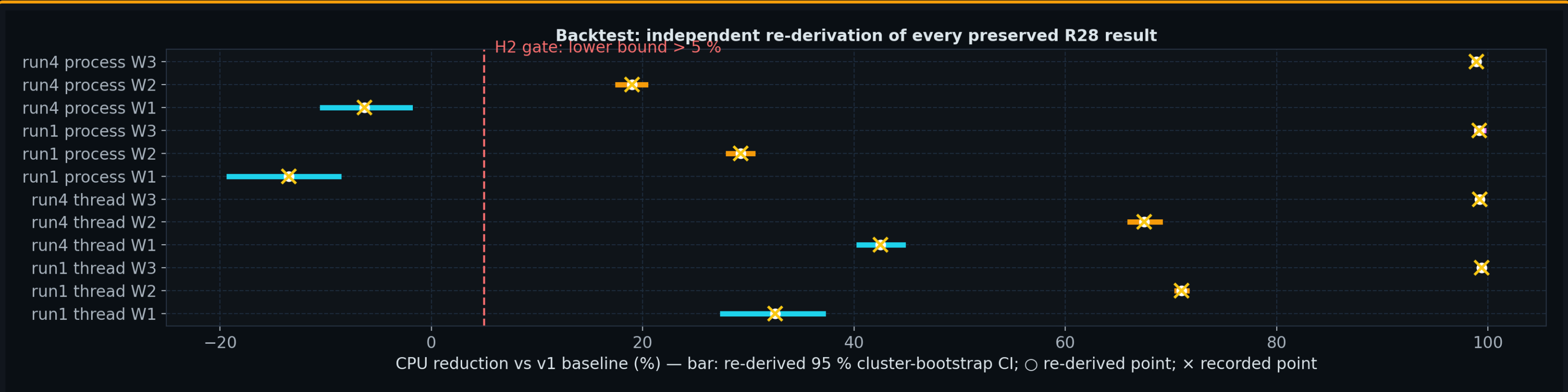


Figure 10.1 — For each of the four preserved result files × three workloads, the note recomputed per-launch mean CPU-seconds per unit from raw.jsonl, then a 10 000-resample cluster bootstrap (seed 28). Every re-derived point (○) sits on the recorded point (x); every interval matches to 3 decimals. The thread candidate clears the 5 % H2 gate on all three workloads; the process ablation fails it on W1.

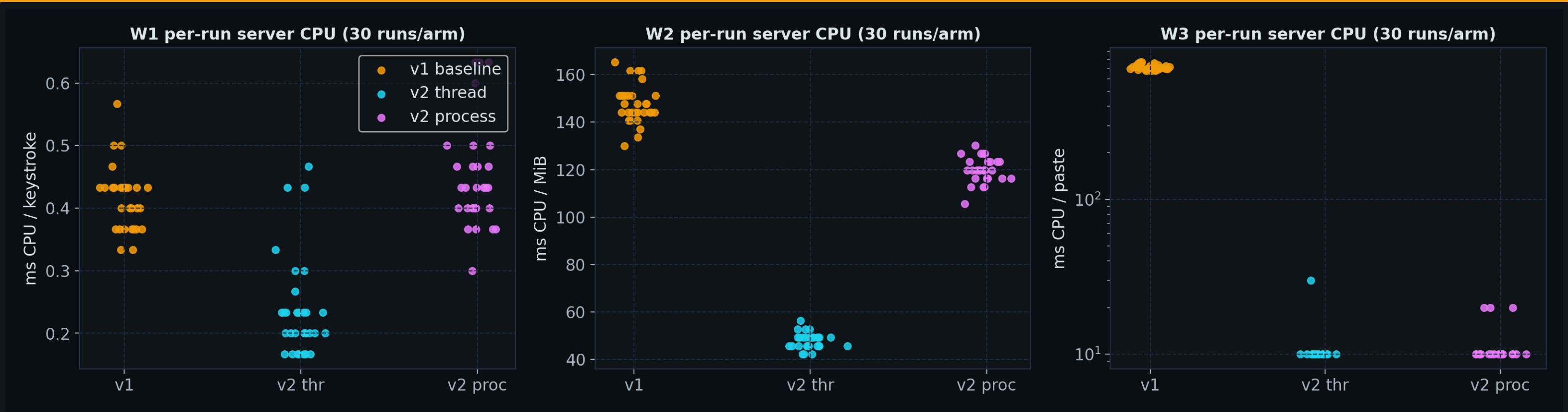


Figure 10.2 — Raw per-run server CPU per unit (reported run 4): 30 warmed runs per arm (3 launches × 10). W1 v1 0.41 → v2 0.24 ms/keystroke; W2 147 → 48 ms/MiB; W3 718 → 6 ms/paste (log axis). All 60 main-arm runs report W2 fidelity "exact".

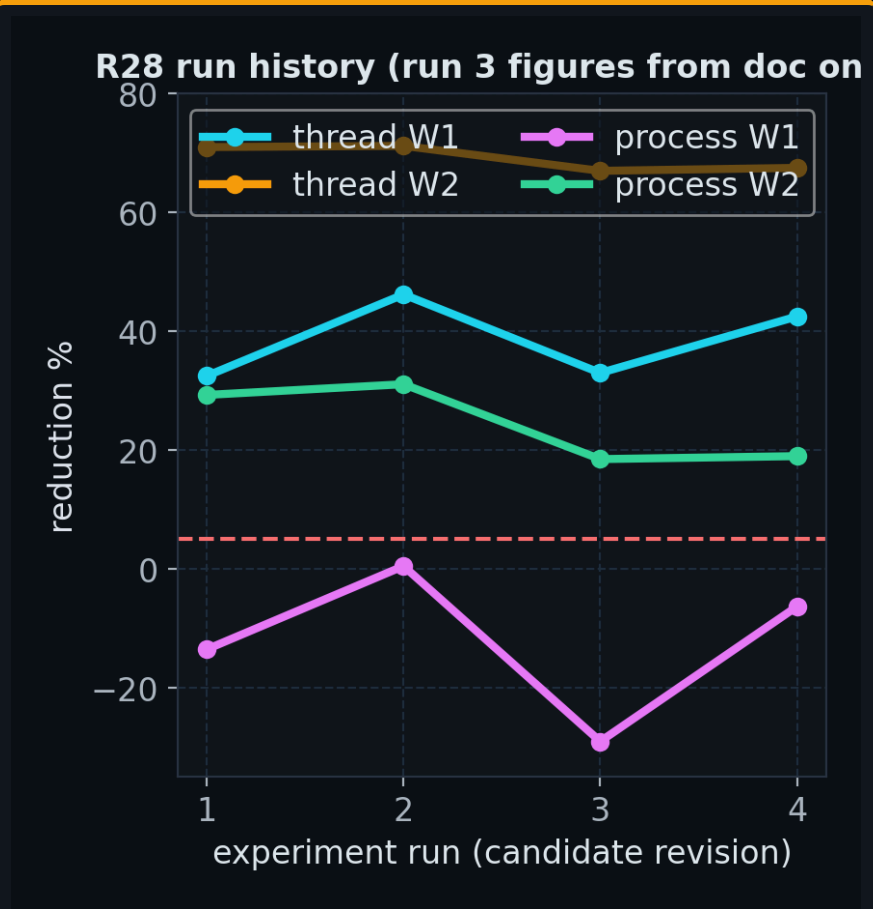


Figure 11.1 — Four candidate revisions, one frozen configuration. Values from the preserved files where they exist; run 3 only in EXPERIMENT.md.

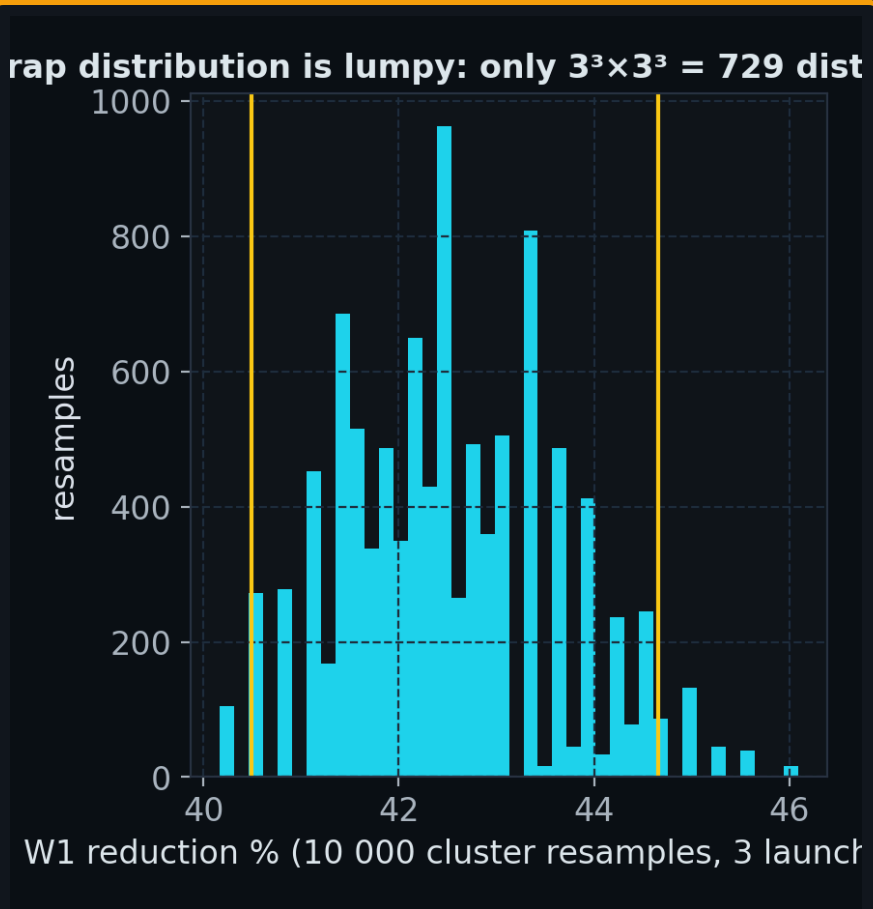


Figure 11.2 — The W1 bootstrap distribution: with 3 launches per arm it is a lattice, not a smooth curve.

11.1 Re-derived table (run 4, thread)

W	v1 per launch	v2 per launch	cut	95 % CI
W1	0.413 / 0.427 / 0.407	0.23 / 0.243 / 0.243	42.5 %	40.5–44.7
W2	146 / 150 / 146	50 / 48.2 / 45.8	67.5 %	66.1–69.0
W3	713 / 723 / 717	6 / 6 / 5	99.2 %	99.2–99.3

Units ms CPU per keystroke / MiB / paste.

11.2 Process ablation

W	cut	95 % CI	H2
W1	-6.3 %	-10.3--2.0	NOT
W2	19.0 %	17.6–20.2	SUPPORTED
W3	98.9 %	98.9–99.0	SUPPORTED

11.3 Discrepancy found

EXPERIMENT.md lists run 1 thread W1 as **46 %**; the preserved **pre-optimisation/result.json** gives **32.5 %** (CI 27.6–37.1). 46.2 % is the *run 2* file. Also, run 1 is stamped 21:01:13Z while the frozen configuration's approval is stamped 21:05:00Z.

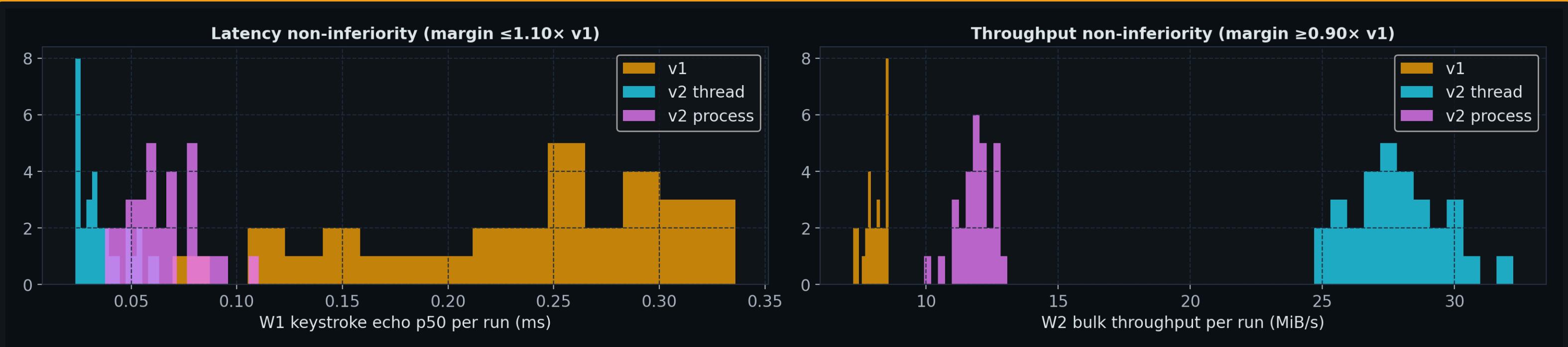


Figure 11.3 — Per-run W1 p50 echo latency and W2 throughput across the 30 runs per arm: v2 thread is far inside both non-inferiority margins (p50 ≈ 0.036 ms vs 0.21–0.27 ms; 27.5–28.3 vs 7.8–8.2 MiB/s).

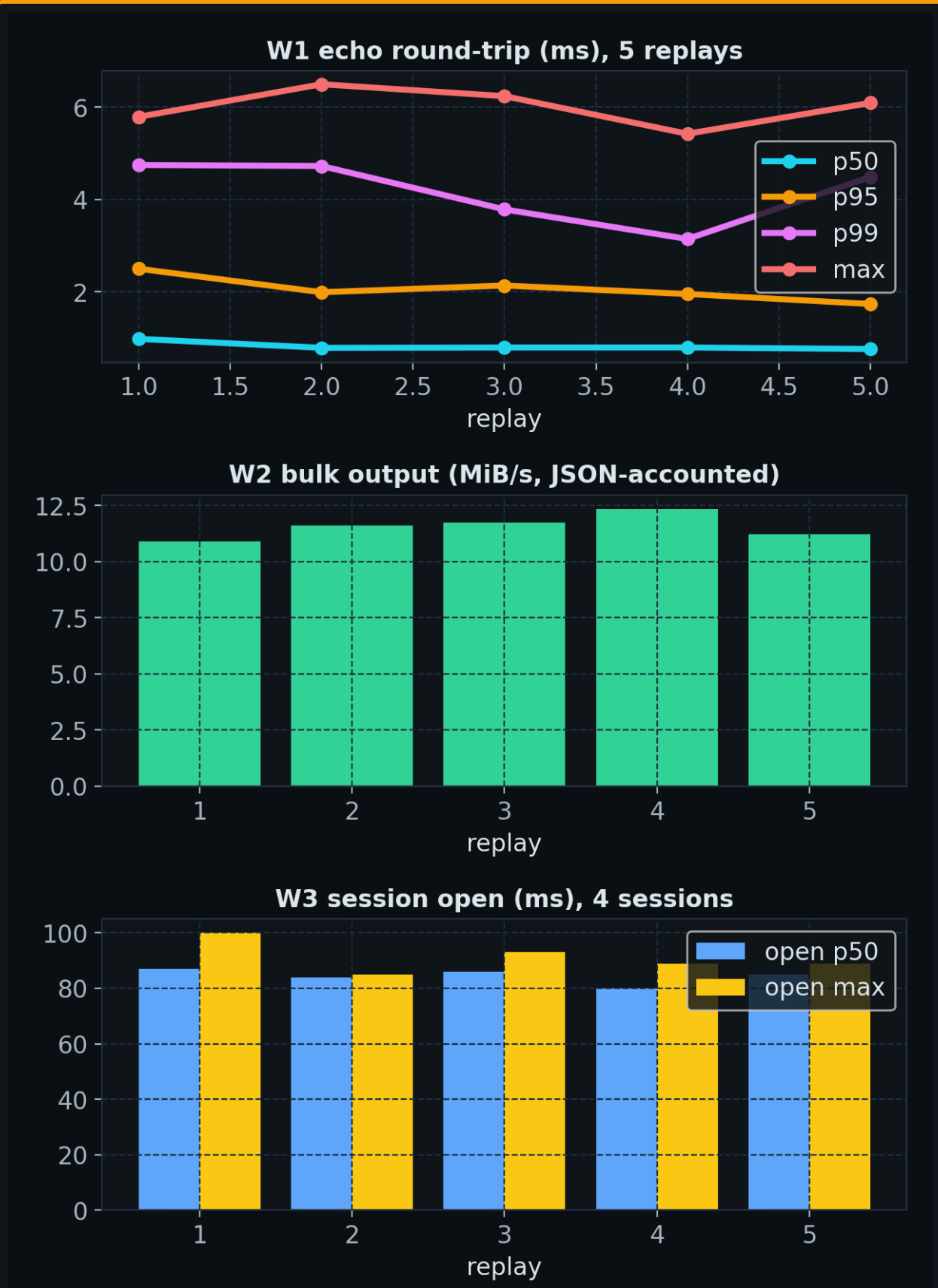


Figure 12.1 — tests/load/bench.js replayed 5× with BENCH_SESSIONS=4 (OBSERVED-HERE, same host class as R28). Note this bench measures the in-process gateway incl. harness; it is not the R28 measurement boundary.

12.1 Replay figures

Replay	W1 p50	p95	p99	W2 MiB/s	W3 open p50 / max
1	0.985	2.502	4.749	10.89	87 / 100 ms
2	0.791	1.994	4.726	11.6	84 / 85 ms
3	0.798	2.14	3.787	11.74	86 / 93 ms
4	0.799	1.956	3.147	12.35	80 / 89 ms
5	0.765	1.74	4.489	11.22	85 / 89 ms

12.2 Finding — the bench fails at its own default

With the default `BENCH_SESSIONS=8` the bench aborts at W3 ("non-101 status"). It raises `maxSessions` and `maxSessionsPerPrincipal` to `S` but not `maxConnectionsPerPrincipal` (default 4, gateway/config.js:85), and all 8 clients use principal `alice`. Fix: pass `maxConnectionsPerPrincipal: S` in `startGateway`.

12.3 Finding — worker RSS unmeasured

W3 reports `workerRssMiB.samples: 0` on the default thread backend: threads have no pid, so the `/proc` read finds nothing. The field is only meaningful with `VWS_WORKER_BACKEND=process`.

12.4 Reading

W2 "MiBps" here is ~11 MiB/s vs R28's 28 MiB/s because the bench streams 400 000 lines through the harness client in-process; the numbers are not comparable and the file itself says "nothing here is a capacity claim".

12.5 Replay spread (5 runs)

Metric	min	max	spread
W1 p50 ms	0.765	0.985	29 %
W1 p99 ms	3.147	4.749	51 %
W2 MiB/s	10.89	12.35	13 %
W3 open p50 ms	80	87	9 %
gateway RSS with sessions MiB	144.6	167.8	16 %

12.6 Replay vs recorded experiment

Quantity	R28 (recorded)	Replay (here)	Comparable?
keystroke echo	0.035-0.037 ms (server-side, separate client process)	0.77-0.99 ms (round-trip incl. in-process harness)	no — different boundary
bulk throughput	27.5-28.3 MiB/s	10.9-12.4 MiB/s	no — 2× output, in-process client
fidelity	exact, 60/60	BULK-END reached 5/5	yes (qualitative)

The replay confirms the transport is stable run-to-run on this host (W1 p50 spread shown above) and that the harness-level defect F3 is reproducible on every attempt at `S = 8`.

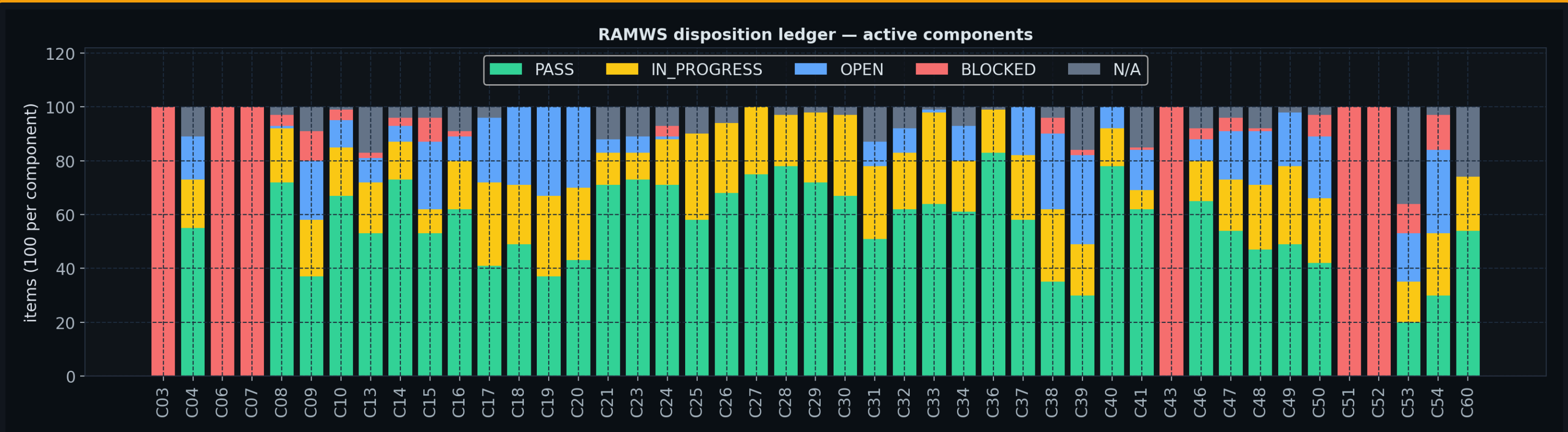
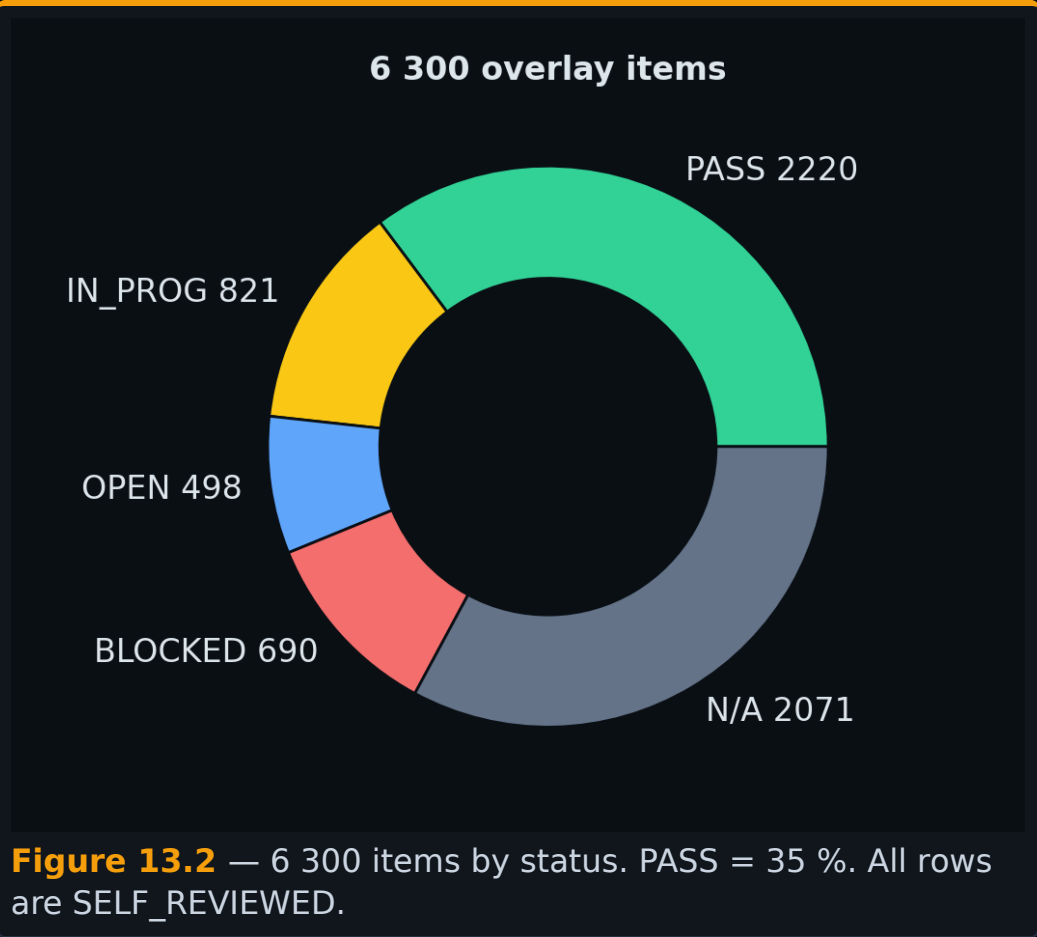


Figure 13.1 — ledger/DISPOSITIONS_SUMMARY.md: per active component, 100 overlay items each. Fully blocked: C03, C06, C07, C43, C51, C52 (PowerShell/.NET/Docker/second host).



13.1 Native work packages e/native/R01-R32

R01	R02	R03	R04	R05	R06	R07	R08
R09	R10	R11	R12	R13	R14	R15	R16
R17	R18	R19	R20	R21	R22	R23	R24
R25	R26	R27	R28	R29	R30	R31	R32

25 PASS · 4 IN_PROGRESS (R05, R07, R14, R20) · 2 N/A (R24, R32) · 1 BLOCKED (R26 Render deploy). ledger/release.json is built from these.

13.2 Finding — self-contradicting summary

The summary's prose says the owner chose "a full per-item re-audit" and that active items were "audited one by one", but its own basis line reads **"Basis: 0 per-item audited, 2400 component-level"**. 2 400 is also not the number of component-level rows implied by the table (24 fully-N/A/BLOCKED components = 2 400 — the per-item count 3 900 is missing). The generator in tools/build_evidence.py should be checked.

13.3 Movement

154 IN_PROGRESS → PASS and 174 N/A → PASS, against 151 PASS → IN_PROGRESS: the re-audit demoted nearly as much as it promoted, a healthy sign.

14.1 Findings of this note

#	Sev.	Finding	Evidence	Fix
F1	Med	EXPERIMENT.md run-1 thread W1 = 46 %; preserved file = 32.5 %	e/R28/pre- optimisation/result.json	correct table row
F2	Low	Run 1 timestamp 21:01Z precedes freeze approval 21:05Z	experiment.frozen.json vs result.json	explain or re-stamp; pre- registration integrity
F3	Med	load bench aborts at default S=8 (connection cap 4)	tests/load/bench.js · config.js:85	set maxConnectionsPerPrincipal
F4	Low	bench worker RSS always empty on thread backend	bench.js rssKb(pid)	report isolate heap via /live
F5	Med	Dispositions summary: "per-item re-audit" vs "0 per- item audited"	DISPOSITIONS_SUMMARY.md	regenerate basis line
F6	High (repro)	containership ship/, hull/, hold/, pk/, edge/evidence absent; uc.py fails; newest gate record is UC-2.1.3 while release is UC-2.8.0	uc.py:20 · UC_GATE_RESULTS.json	ship the ship layer or document where it lives; re- run gates on 2.8.0

14.2 Confirmed strengths

Reproducible statistics — every preserved R28 number re-derives exactly. **Honest claims** — "RAM-resident" is defined narrowly, H1/H4 are marked NOT RUN, blocked work is never PASS. **Robust framing** — no crash across 5 000 randomized decoder inputs. **Ledger invariants** hold under a 5× longer walk than shipped.

14.3 Residual risk register (project + this note)

Risk	L	I	Mitigation in tree
in-process worker isolates share gateway address space	M	H	process backend + container per gateway
/live unauthenticated	M	L	restrict at edge
no human security review	H	M	two AI adversarial passes (R2-1 fixed)
single-host evidence (3 clusters)	H	M	stated in EXPERIMENT.md
limits "PROPOSED", not approved	M	M	owner approval pending
token in ACCESS_TOKEN.txt on disk	L	M	gitignored; SHA-256 at rest in principals.json

14.4 Recommendations (ordered)

- Fix F3 and F1, re-publish EXPERIMENT.md; attach the run-3 raw files if they can be regenerated.
- Commit or externally pin the containership ship layer so `uc.py verify --quick` runs from a clone; re-run the 19 gates on UC-2.8.0.
- Add P1-P4 to `npm test`; they cost 0.46 s.
- Run H1 on the process backend where RSS is per session.
- Obtain one independent human review of gateway/ws.js and connection.js before any non-loopback exposure.

14.5 What would change the verdict

A failing re-run of the 6 DF fabric tests with DF_ROOT attached; a UC-2.8.0 gate run that does not reproduce the UC-2.1.3 PASS; or the (2) zip containing transport changes — each would require this note to be reissued. None of the six findings touches the claims of Figures 10.1-11.3.

14.6 Severity scale

High blocks reproduction of a stated result · **Med** misstates a number or breaks a shipped tool · **Low** bookkeeping or observability gap. L / I in 14.3 = likelihood / impact (L, M, H).

A.1 config/limits.json (verbatim values)

Key	Value	Key	Value
envelopeBytes	65536	seqItems	1000000
bridgeBodyBytes	65536	regexChars	256
inputBytesPerMessage	8192	regexLineChars	16384
outputChunkBytes	16384	scrollbackRows	5000
sendQueueBytes	524288	scrollbackCells	2000000
controlReserveBytes	16384	oscUnits	4096
ackWindowBytes	8388608	titleChars	256
workerPendingOutputBytes	4194304	csiParamChars	64
replayRetention	not implemented (resume=false)	csiParamValue	65535
vfsBytes	16777216	maxSessions	4
vfsNodes	1000	maxSessionsPerPrincipal	2
pipelineCaptureChars	1048576	messagesPerSecond	300
inputLineChars	4096	reconnect	uniform(0, min(30000, 500 * 2^attempt) max 8 attempts, fresh ticket each time, no input replay
historyEntries	1000	maxConnectionsPerPrincipal	4
		maxPauseMs	120000
		workerStdinBytes	1048576
		controlFramesPerSecond	200
		ticketFailuresPerAddressPerMinute	60
		ticketsPerPrincipalPerMinute	60

A.2 Timeouts (ms)

Timer	ms
handshake	10000
open	10000
assembly	5000
heartbeatInterval	20000
heartbeatWindow	10000
workerPing	5000
workerStall	15000
workerReady	8000
idle	1800000
closeHandshake	3000
ticketTtl	30000
shutdownBudget	20000

A.3 Fabric

slots 2 · lease wait 60 000 ms · deadlines run 180 000 / verify 900 000 / build 900 000 ms.

A.4 Reproduce this note

```
cd bridge-terminal
node scripts/syntax-check.js      # 86 ok
npm test                          # 120 pa
node --test tests/supplementary/  # P1-P5
BENCH_SESSIONS=4 node tests/load/bench.js
python3 backtest.py              # re-der
```

A.5 Host

Linux 6.18.44-fc-v37 x86_64, 2 vCPU Intel Xeon @ 2.80 GHz, Node v22.22.2, Python 3 — the same host class the project's own R28 record names.

A.6 Identity

candidate 2.0.0-ramws.1 · kit RAMWS v1.0.0
4bf7b67e... · baseline hermit-vws 1.1.0-vws.1
db65b5e3... · UC-2.8.0 · edge atoms 86.